

# Neural Networks

Total points 19/19

The respondent's email (aaradhyadevtmr@gmail.com) was recorded on submission of this form.

✓ A student trains the same 2-layer network three times with SGD, SGD+momentum(0.9), and Adam, all with the same learning rate of 0.01. Adam's loss explodes to NaN within 3 epochs while the other two train fine. What is the most likely cause? \*1/1

- ☐ Adam is fundamentally less numerically stable than SGD
- ☒ A learning rate of 0.01 is reasonable for SGD but often too high for Adam, since Adam's adaptive per-parameter scaling can amplify effective step size in directions with small accumulated gradient variance ✓
- ☐ The model must contain a bug in its forward pass that only Adam exposes
- ☐ NaN losses with Adam always indicate a data normalization problem

✓ A model trains with Adam for 15 epochs. Training loss is smooth and steadily decreasing. Validation loss decreases until epoch 6, then plateaus and stays flat (not rising) through epoch 15. What does this pattern most likely indicate? \*1/1

- ☐ Severe overfitting requiring immediate early stopping
- ☐ The model has converged to a stable underfitting regime, lacking capacity to keep improving
- ☒ The model is still learning representations useful for training data, but has saturated its ability to generalize further at the current capacity/data/regularization, more epochs alone won't help, but this is milder than classic overfitting since val loss isn't rising ✓
- ☐ A bug in the validation loop is freezing the reported value



✓ Two CNNs, identical except one uses He init and one uses Xavier init, both with ReLU throughout. After 1 epoch, the He-initialized model has higher early-layer activation variance and trains faster. Why? \*1/1

- ☒ Xavier's  $1/\text{fan\_in}$  scaling assumes a symmetric, zero-mean activation distribution (true for tanh/sigmoid), but ReLU zeroes out half the input distribution, so Xavier under-corrects for that and effectively shrinks signal through layers; He's  $2/\text{fan\_in}$  compensates ✓
- ☐ He initialization sets a higher effective learning rate for the first epoch, which is why it appears to train faster before settling into the same convergence rate as Xavier
- ☐ Xavier initialization was designed specifically for convolutional layers, while He was designed for fully connected layers, so applying Xavier in a CNN context is a mismatch unrelated to the activation function
- ☐ The variance difference is a normal, expected artifact of random initialization and isn't actually attributable to which init scheme was used, since both are mathematically equivalent for ReLU networks

✓ A  $200 \times 200$  grayscale image passes through  $\text{Conv2d}(1,16,k=3,\text{pad}=1) \rightarrow \text{MaxPool2d}(2) \rightarrow \text{Conv2d}(16,32,k=3,\text{pad}=1) \rightarrow \text{MaxPool2d}(2)$ . What's the output spatial size? \*1/1

- ☐  $196 \times 196$ , since each conv with a  $3 \times 3$  kernel always trims 2 pixels per side regardless of padding settings
- ☒  $50 \times 50$ , since padded  $3 \times 3$  convs preserve size while each pool halves it:  $200 \rightarrow 200 \rightarrow 100 \rightarrow 100 \rightarrow 50$  ✓
- ☐  $25 \times 25$ , since both pooling and convolution operations are assumed to halve spatial size by default
- ☐  $100 \times 100$ , since one pooling stage halves the image and the second conv cancels the other pool's effect



✓ "Lower cross-entropy loss for Model A than Model B means Model A has higher accuracy." Evaluate this. \*1/1

- ☐ True, since cross-entropy and accuracy track each other exactly whenever the test set and class count match
- ☐ False, since accuracy can only be compared between models trained for an identical number of epochs
- ☒ False, since cross-entropy reflects prediction confidence, so A could be lower-loss yet still misclassify more examples than the less confident but more often correct B ✓
- ☐ True, but only if both models share the same optimizer, since different optimizers calibrate loss differently

✓ Adding BatchNorm2d after every Conv2d and Dropout before the final layer simultaneously performs worse than either alone. Why? \*1/1

- ☐ BatchNorm and Dropout cannot coexist in the same module and one silently fails to apply
- ☐ Combining any two regularizers always tips a model from overfitting into underfitting territory
- ☐ This combination is a known incompatibility that PyTorch documentation explicitly warns against using together
- ☒ Dropout's training-time noise shifts the activation distribution BatchNorm normalizes against, so the mismatch between training-time and inference-time statistics can degrade BatchNorm's learned parameters ✓



✓ NEU is small (1,800 images), so skip augmentation." Evaluate. \* 1/1

- ☐ This is correct
- ☒ Incorrect - small datasets benefit more
- ☐ Option 3
- ☐ Only true for grayscale images
- ☐ True only above 5,000 images



✓ 95% overall accuracy but inclusion-defect recall is 60%, balanced test set. \*1/1  
Best response?

- ☐ Collect more inclusion images
- ☐ Lower the overall accuracy target
- ☒ Report also per class recall
- ☐ Retrain with a smaller batch size



✓ Why does gradient clipping preserve direction while bounding magnitude? \*1/1

- ☒ Scale whole gradient vector by one factor
- ☐ Clips each weight directly
- ☐ It modifies the loss function before backprop
- ☐ Direction isn't actually preserved



✓ Why use cosine annealing instead of a constant low learning rate? \* 1/1

- ☐ Constant LR is simply slower, nothing else differs
- ☐ PyTorch doesn't support constant LR
- ☒ Due to high LR that helps escape poor minima ✓
- ☐ Cosine annealing avoids the need for backpropagation

✓ RandomRotation(15) suits steel defects; RandomRotation(90) could hurt \*1/1  
digit recognition. Why?

- ☐ Compute cost
- ☒ A rotated "6" can resemble a different digit, changing its true label ✓
- ☐ MaxPool2d misalignment
- ☐ Digits are grayscale

✓ Switching ReLU to Sigmoid slows training dramatically. Why? \* 1/1

- ☐ Sigmoid caps the loss value
- ☐ Sigmoid is incompatible with Adam
- ☒ Due to saturated sigmoids (near zero gradients) ✓
- ☐ The learning rate is too high which makes convergence slower



✓ "Dropout reduces capacity, so it should never outperform no-dropout." \*1/1  
Correct?

- ☐ True
- ☒ False



✓ Why does Huber Loss combine MSE-like and MAE-like behavior? \* 1/1

- ☐ Pure compute efficiency
- ☒ Outlier influence is limited
- ☐ It's identical to MSE everywhere
- ☐ MAE is smoother than MSE near zero



✓ Accuracy drops from 90% to 70% in production due to drift in lighting/angle/texture. Best fix? \*1/1

- ☐ Train more epochs
- ☒ Retrain on production data
- ☐ Switch from the current optimiser
- ☐ Raise the learning rate



✓ Same setup, different seeds, gives 84% vs. 79%. What does this imply about a single-run comparison claiming "BatchNorm gives +5%"? \*1/1

- ☐ Match the seeds and the problem disappears
- ☐ Irrelevant once BatchNorm is used
- ☐ Indicates a bug
- ☒ The gap may be seed noise ✓

✓ Adding 3 conv layers lowers validation accuracy while training accuracy stays high. Why, and is more depth generally safe for small data? \*1/1

- ☐ Larger receptive field directly hurts accuracy
- ☐ Exploding gradients
- ☐ Extra capacity overfits the small dataset; depth should match data volume
- ☒ Vanishing gradients, which is inconsistent with steady training accuracy ✓

✓ Why normalize val/test data using training-set statistics, not their own? \* 1/1

- ☐ Computational cost
- ☒ Different stats shift the input distribution relative to what the model was trained on ✓
- ☐ Only matters with BatchNorm
- ☐ PyTorch requires it



- ✓ Validation loss is jagged but flat overall while training loss is smooth. \*1/1  
Why, and is it a problem?
- ☐ Data leakage between splits
  - ☐ LR decaying too aggressively
  - ☒ Small validation set (not really a problem) ✓
  - ☐ An unavoidable Adam artifact

This form was created inside fusemachines. - [Contact form owner](#)

Does this form look suspicious? [Report](#)

Google Forms





